Абстрактни класове и интерфейси

титуляр на курса: д-р Тодор Цонков (ttsonkov@gmail.com)

практически примери

теория

От какво ще се състои настоящата лекция?

Абстрактни класове и интерфейси. Pure virtual functions. Object slicing, type casts. Type Erasure (std::function, C++ 11). Сравнение с езика Java.

Какво е абстрактен клас?

Абстрактен клас:

Не може да се инстанцира
Служи като интерфейс + базова логика
Има поне една pure virtual функция

Гарантира, че всички наследници:
Имплементират нужните функции и спазват едно и също API.

Може да има и член данни за разлика от интерфейса!

Синтаксис

```
class Person {

protected:

    std::string name_;

public:

    Person(std::string name) :
name_(std::move(name)) {}

    virtual void print() const {

        std::cout << "Name: " << name_ << "\n";

    }

    virtual std::string role() const = 0;

};
```

Какво е интерфейс

В C++ няма ключова дума interface.

Интерфейсът е дизайнерска концепция, която се реализира чрез:

клас, който съдържа само pure virtual функции
без състояние (data members)
с виртуален деструктор
Целта е да дефинира какво може обектът, а не как е реализиран.

Следва Dependency Inversion Principle (DIP)

Синтаксис

```
class Person {

public:

    virtual ~Person() = default; //
ЗАДЪЛЖИТЕЛНО

    virtual std::string role() const = 0; //
pure virtual

};

class Student : public Person {

public:

    std::string role() const override {

        return "Student";

    }

};
```

Object Slicing

Получава се при:
Копиране по стойност
Базовият клас не знае за допълнителните членове

Решение: указатели или референции

```
void process(const Person& p);   // OK
void process(Person* p);         // OK
```

❌ Никога: `void process(Person p);`

Синтаксис

```
class Person {
public:
    std::string name;
};

class Student : public Person {
public:
    int facultyNumber;
};

Person p = Student{"Ivan", 123}; // slicing!
```

Type casts в езика C++

Обяснение

Type cast = изрично преобразуване на тип.В C++ има 4 различни оператора, всеки със строго определено предназначение. Различните cast-ове имат различна степен на безопасност.

Пример

Student* s = (Student*)p; //
1)НЕ може да означава static_cast, reinterpret_cast или const_cast

2)няма яснота какво реално прави
3)трудно се ревюира
4)прикрива грешки

static_cast

double d = 3.14;
int i = static_cast<int>(d);
✔ проверява се по време на компилация
✔ без runtime overhead
❌ не проверява реалния тип на обекта

Type casts в езика C++

Проблем

```
Person* p = new Student{};
Student*s=static_cast<Student*
>(p);
```

Опасно, ако p не сочи към Student

Решение

```
Person* p = new Student{};

Student* s =
dynamic_cast<Student*>(p);
if (s) {
   // безопасно
}
```

проверява реалния тип
връща nullptr при грешка
runtime overhead

dynamic_cast

Класът трябва да е polymorphic (да има поне една virtual функция

```
class Person {
public:
   virtual ~Person() = default;
};
```

const_cast

const_cast е единственият cast в C++, който:
може да премахва (const) и (volatile) квалификатори
2)не променя типа, само cv-квалификациите
const int x = 10;
int& y = const_cast<int&>(x);
3) Това не означава, че е безопасно да се модифицира x.

🔹 Какво МОЖЕ да прави const_cast

✔ маха const / volatile
✔ добавя const (рядко полезно)
✔ работи с указатели и референции

reinterpret_cast

reinterpret_cast е най-ниско-нивовият и най-опасният cast в C++.

➡️ Казва на компилатора:

„Интерпретирай тези битове по друг начин.“
int* p = reinterpret_cast<int*>(0x12345678);

Какво може да прави:

✔ преобразува между несвързани типове указатели
✔ pointer ↔ integer
✔ function pointer ↔ function pointer

🔹 Златно правило

Ако мислиш, че ти трябва reinterpret_cast, 99% от времето дизайнът е грешен.

Какво е type erasure?

Type Erasure е техника, при която:

Скриваме конкретния тип на обект
Работим с него чрез унифициран интерфейс
👉 Идея: „Не ме интересува какъв е типът, важно е какво може да прави“

Цели:
Полиморфизъм без наследяване
Runtime гъвкавост

Какъв проблем решава?

```
template<typename F>
void run(F f) {
    f();
}
// Ще разгледаме темплейтите в детайли в следваща лекция
```

❌ Всеки различен F → нова инстанция
❌ Няма общ тип за съхранение
❌ Трудно предаване между модули

👉 Не можем да кажем:

```
std::vector<??> tasks;
```

std::function

std::function:

Приема всякакъв callable обект:
функция, lambda, functor, std::bind

obj(args...);
=> държи се като функция

Скрива реалния тип → type erasure
Осигурява един общ тип

Какво е std::bind?

```
int add(int a, int b) {
    return a + b;
}

auto add5 = std::bind(add, 5,
std::placeholders::_1);
add5(3);   // 8
```

Примери

```
void foo() { std::cout << "foo\n"; }

struct Bar {
    void operator()() const {
        std::cout << "bar\n";
    }
};

std::function<void()> f1 = foo;

std::function<void()> f2 = [] { std::cout <<
"lambda\n"; };

std::function<void()> f3 = Bar{};
```

Пример предметната област School

```
class Person {
public:
    virtual ~Person() {}
    virtual std::string name() const = 0;   // pure virtual
    virtual void print() const = 0;
};

class Student : public Person {
    std::string m_name;
    double m_gpa;

public:
    Student(std::string name, double gpa)
        : m_name(name), m_gpa(gpa) {}

    std::string name() const override { return m_name; }
    double gpa() const { return m_gpa; }

    void print() const override {
        std::cout << "Student: " << m_name
                  << ", GPA=" << m_gpa << "\n";
    }
};
```

Пример предметната област School

```
class School {
    std::vector<IPerson*> students; // raw pointers (без ownership)

public:
    void add(IPerson* s) {
        students.push_back(s);
    }

    // Type erasure – приема всякакъв callable
    void for_each(const std::function<void(const IPerson&)>& action) const {
        for (const IPerson* s : students)
            action(*s);
    }
};

Използване:
    Student s1("Ivan", 3.4);
    Student s2("Maria", 4.0);

    School school;
    school.add(&s1);
    school.add(&s2);

    // Lambda
    school.for_each([](const IStudent& s) {
        s.print();
    });
```

Пример на Java

```java
public interface Student {
    String getName();
    void printInfo();
}

public class RegularStudent implements Student {
    private String name;
    private double gpa;

    public RegularStudent(String name, double gpa) {
        this.name = name;
        this.gpa = gpa;
    }

    public String getName() {
        return name;
    }

    public double getGpa() {
        return gpa;
    }

    @Override
    public void printInfo() {
        System.out.println("Student: " + name + ", GPA=" + gpa);
    }
}
```

Пример на Java

```java
import java.util.ArrayList;
import java.util.List;
import java.util.function.Consumer;
import java.util.function.Predicate;
public class School {
    private List<Student> students = new ArrayList<>();

    public void add(Student s) {
        students.add(s);
    }

    // Consumer<Student> = еквивалент на std::function<void(Student)>
    public void forEachStudent(Consumer<Student> action) {
        for (Student s : students) {
            action.accept(s);
        }
    }

    public void filterAndApply(Predicate<Student> pred,
                               Consumer<Student> action) {
        for (Student s : students) {
            if (pred.test(s)) {
                action.accept(s);
            }
        }
    }
}
```

Въпроси?

Благодаря за вниманието!